

JAVA OOP LABORATORY

Library Management System

1. Objective

To develop a Library Management System in Java and demonstrate important Object-Oriented Programming concepts such as classes and objects, constructors, encapsulation using private data members with getters and setters, inheritance, method overloading, method overriding, polymorphism, abstraction, and interfaces.

2. Task Description

Develop a Java program that represents a simple library system. The program must create and display book details using constructors and encapsulation. It must also demonstrate inheritance using Person1, Student1, and Faculty1; method overloading using Areas; method overriding and runtime polymorphism using Vehicles, Cars, and Bikes; abstraction using Shapes, Circles, and Rectangles; and interface implementation using Prints and Reports.

3. Step-wise Execution

1. Create the Books class with private fields bookId, bookName, author, and price.
2. Create a default constructor and a parameterized constructor for Books.
3. Use setter methods to assign values to the first book and getter methods through the display method.
4. Create a second Books object using the parameterized constructor.
5. Create Person1 as the parent class and Student1 and Faculty1 as child classes to demonstrate inheritance.
6. Create the Areas class and overload area() for square, rectangle, and circle.
7. Create Vehicles as the parent class and override display() in Cars and Bikes.
8. Create the abstract Shapes class and implement draw() in Circles and Rectangles.
9. Create the Prints interface and implement print() in Reports.
10. Run the main method and display the results of all concepts.

4. Algorithm

1. START
2. Create a Books object using the default constructor.
3. Set book ID, name, author, and price using setter methods.
4. Display the first book details.
5. Create another Books object using the parameterized constructor and display its details.
6. Create Student1 and Faculty1 objects and display inherited Person1 details.
7. Create Areas object and call overloaded area() methods.
8. Create Vehicles reference and assign Cars and Bikes objects; call display() to demonstrate overriding.
9. Create Shapes reference and assign Circles and Rectangles objects; call draw().
10. Create Reports object and call print() from the implemented interface.
11. STOP

5. UML Diagram

Class / Interface	Relationship	Important Members
-------------------	--------------	-------------------

Books	Class	private bookId, bookName, author, price; constructors; getters/setters; display()
Person1	Parent class	name, age
Student1	extends Person1	rollNo, displayStudent()
Faculty1	extends Person1	subject, displayFaculty()
Areas	Class	area(int), area(int,int), area(double)
Vehicles	Parent class	display()
Cars / Bikes	extends Vehicles	overridden display()
Shapes	Abstract class	abstract draw()
Circles / Rectangles	extends Shapes	draw() implementation
Prints	Interface	print()
Reports	implements Prints	print() implementation
Library	Main class	main()

UML relationship summary: Person1 is the superclass of Student1 and Faculty1. Vehicles is the superclass of Cars and Bikes. Shapes is an abstract superclass of Circles and Rectangles. Prints is implemented by Reports. Library uses objects of all these classes.

6. Program Code

```
class Books {

    private int bookId;
    private String bookName;
    private String author;
    private double price;
    public Books() {
        System.out.println("Default Constructor Called");
    }
    public Books(int id, String name, String author, double price) {
        this.bookId = id;
        this.bookName = name;
        this.author = author;
        this.price = price;
    }

    //getters Setters
    public void setBookId(int id) {
        bookId = id;
    }

    public void setBookName(String name) {
        bookName = name;
    }

    public void setAuthor(String a) {
        author = a;
    }

    public void setPrice(double p) {
        price = p;
    }

    // Getters
    public int getBookId() {
        return bookId;
    }

    public String getBookName() {
        return bookName;
    }

    public String getAuthor() {
        return author;
    }

    public double getPrice() {
        return price;
    }

    public void display() {
        System.out.println("Book ID : " + bookId);
        System.out.println("Book Name : " + bookName);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
    }
}

class Person1 {
    String name = "priya";
    int age = 22;
}

class Student1 extends Person1 {
    int rollNo = 156;

    void displayStudent() {
        System.out.println("\nStudent Details");
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Roll No : " + rollNo);
    }
}

class Faculty1 extends Person1 {
    String subject = "Java";

    void displayFaculty() {
        System.out.println("\nFaculty Details");
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Subject : " + subject);
    }
}

class Areas {

    // Method Overloading
```

```

    void area(int side) {
        System.out.println("\nArea of Square = " + (side * side));
    }

    void area(int length, int breadth) {
        System.out.println("Area of Rectangle = " + (length * breadth));
    }

    void area(double radius) {
        System.out.println("Area of Circle = " + (3.14 * radius * radius));
    }
}

// Method Overriding

class Vehicles {
    void display() {
        System.out.println("\nThis is a Vehicle");
    }
}

class Cars extends Vehicles {
    void display() {
        System.out.println("This is a Car");
    }
}

class Bikes extends Vehicles {
    void display() {
        System.out.println("This is a Bike");
    }
}

abstract class Shapes {
    abstract void draw();
}

class Circles extends Shapes {
    void draw() {
        System.out.println("\nDrawing Circle");
    }
}

class Rectangles extends Shapes {
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}

interface Prints {
    void print();
}

class Reports implements Prints {
    public void print() {
        System.out.println("\nPrinting Library Report");
    }
}

public class Library {

    public static void main(String[] args) {
        Books b1 = new Books();

        b1.setBookId(167);
        b1.setBookName("Java Programming");
        b1.setAuthor("James Gosling");
        b1.setPrice(599);

        System.out.println("\nBook Details");
        b1.display();

        Books b2 = new Books(102, "Python", "Guido", 700);

        System.out.println("\nParameterized Constructor");
        b2.display();
        Student1 s = new Student1();
        s.displayStudent();

        Faculty1 f = new Faculty1();
        f.displayFaculty();

        // Method Overloading
        Areas a = new Areas();
        a.area(6);
        a.area(10, 24);
        a.area(3.7);

        // Method Overriding
        Vehicles v;

        v = new Cars();
    }
}

```

```
v.display();

v = new Bikes();
v.display();
Shapes sh;

sh = new Circles();
sh.draw();

sh = new Rectangles();
sh.draw();
Reports r = new Reports();
r.print();
}
```

7. Output Screen

```
Default Constructor Called

Book Details
Book ID : 167
Book Name : Java Programming
Author : James Gosling
Price : 599.0

Parameterized Constructor
Book ID : 102
Book Name : Python
Author : Guido
Price : 700.0

Student Details
Name : priya
Age : 22
Roll No : 156

Faculty Details
Name : priya
Age : 22
Subject : Java

Area of Square = 36
Area of Rectangle = 240
Area of Circle = 43.0106

This is a Car
This is a Bike

Drawing Circle
Drawing Rectangle

Printing Library Report
```

8. Result

The Library Management System was implemented successfully in Java. The program demonstrates constructors, encapsulation, inheritance, method overloading, method overriding, runtime polymorphism, abstraction, and interfaces with the required output.

9. Conclusion

Thus, the Java program successfully demonstrates the major Object-Oriented Programming concepts required for the Library Management System.